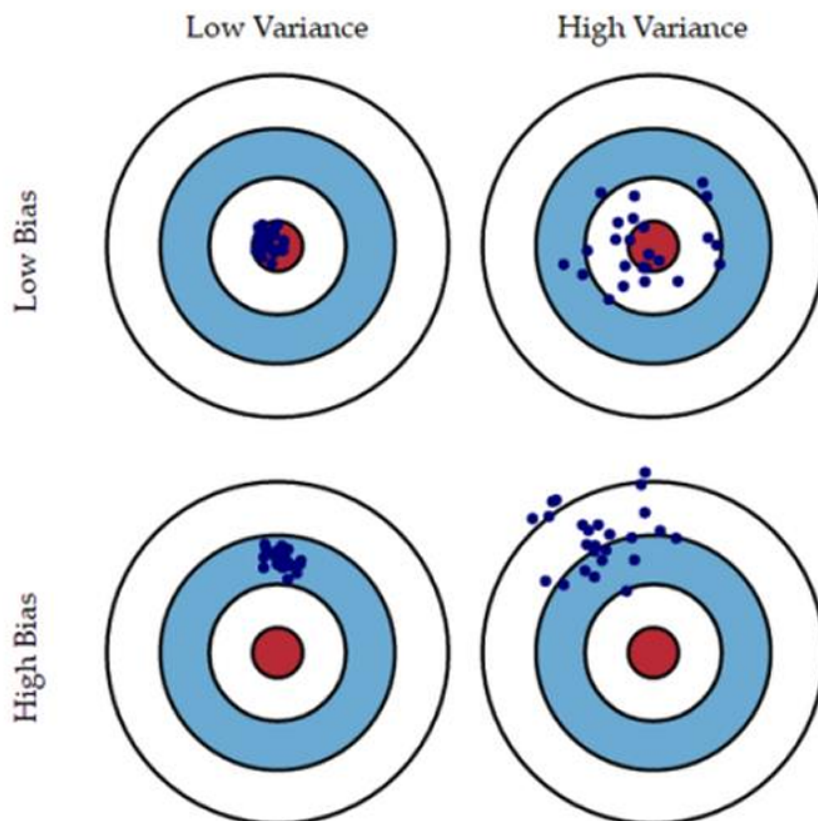


- [1. Bagging](#)
 - [1.1 核心思想](#)
 - [1.2 二元分类](#)
 - [1.3 相关性的影响](#)
 - [1.4 随机森林 \(Random Forest\)](#)
 - [1.5 小结](#)
- [2. Boosting](#)
 - [2.1 AdaBoost 算法](#)
 - [2.1.1 弱学习器 \(Weak Learner\)](#)
 - [2.1.1.1 决策桩 \(Decision Stumps\)](#)
 - [2.1.2 AdaBoost 算法的具体步骤](#)
 - [2.1.2.1 AdaBoost 算法中权重调整](#)
 - [2.1.3 AdaBoost 算法的示例](#)
 - [2.1.4 最小化训练误差](#)
 - [2.1.5 AdaBoost 算法的泛化误差 \(generalization error\)](#)
 - [2.1.6 作为拟合加性模型 \(Additive Models\)](#)
 - [2.1.6.1 指数损失函数](#)
 - [2.1.7 AdaBoost 算法小结](#)
 - [2.2 小结](#)
- [3. 集成学习](#)

1. Bagging

我们上一章说到偏差方差分解。

下图用飞镖的比喻来形象地解释了偏差 (Bias) 和方差 (Variance) 对模型预测性能的影响。



四个飞镖靶，每个靶上都有不同分布的飞镖点，代表模型在不同情况下的预测结果。

左上角（低偏差，低方差）：飞镖点集中在靶心附近，表示模型预测准确且稳定。

右上角（低偏差，高方差）：飞镖点分布分散但都靠近靶心，表示模型预测稳定但不够准确。

左下角（高偏差，低方差）：飞镖点集中在偏离靶心的位置，表示模型预测不准确但稳定。

右下角（高偏差，高方差）：飞镖点分散且偏离靶心，表示模型预测既不准确也不稳定。

我们现在讨论如何优化偏差方差。

我们可以训练多个模型，每个模型给出一个预测值 Y_i ，然后把它们的预测结果取平均，得到最终预测

$$\bar{y} = \frac{1}{m} \sum_{i=1}^m Y_i$$

这对我们模型的“期望损失”三项（Bayes误差、偏差、方差）有什么影响？

Bayes误差（不可约误差）不变，因为这是数据本身的噪声，我们做什么模型都改不了。

偏差不变，因为每个模型的期望预测值是一样的，平均之后期望也没变。

$$\mathbb{E}[\bar{y}] = \mathbb{E}\left[\frac{1}{m} \sum_{i=1}^m Y_i\right] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}[Y_i]$$

换句话说，Bagging 不会让一个“错的模型”变成对的，它只是把多个“同样偏”的模型平均一下。

方差减少了，因为我们在对多个独立同分布的预测结果做平均。

$$\text{Var}(\bar{y}) = \text{Var}\left(\frac{1}{m} \sum_{i=1}^m Y_i\right) = \frac{1}{m^2} \sum_{i=1}^m \text{Var}(Y_i) = \frac{1}{m} \text{Var}(Y_i)$$

平均独立随机变量会让方差按 $1/m$ 的比例下降。

1.1 核心思想

在实际中，数据分布 p_{sample} 往往是有限的或采样成本很高，所以每次都从真实分布中独立采样新数据集来训练模型是非常浪费数据的。

那为什么不只用一个数据集，然后通过有放回地重复采样来“模拟”多个数据集呢？

这就是 Bagging (Bootstrap Aggregating) 的核心思想。

你有一个大小为 n 的训练集 D 。

从这个数据集中有放回地随机采样 n 个样本，生成一个新的数据集（也叫 bootstrap 样本）。

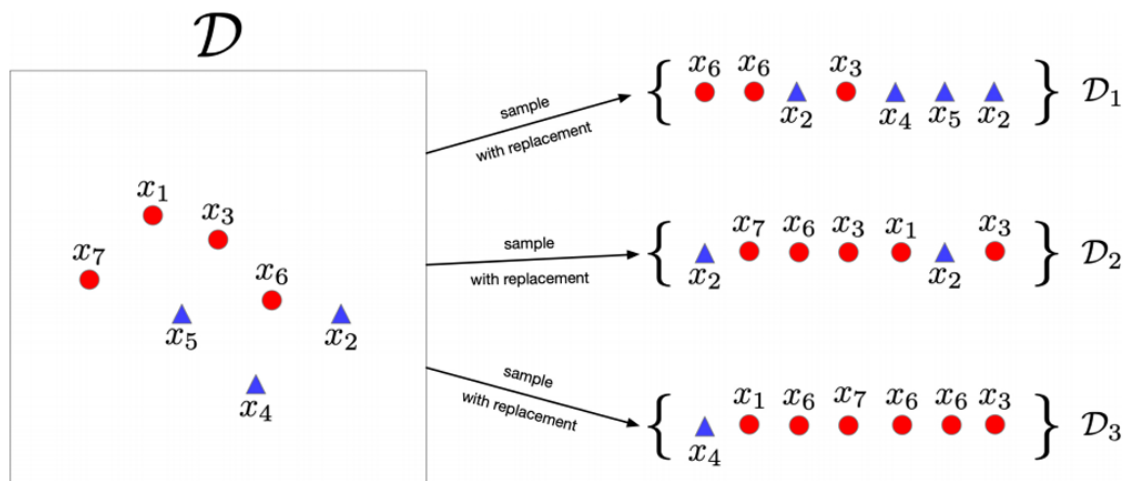
重复这个过程 m 次，得到 m 个“新”数据集。

在每个数据集上训练一个模型，最后平均（或投票）它们的预测结果。

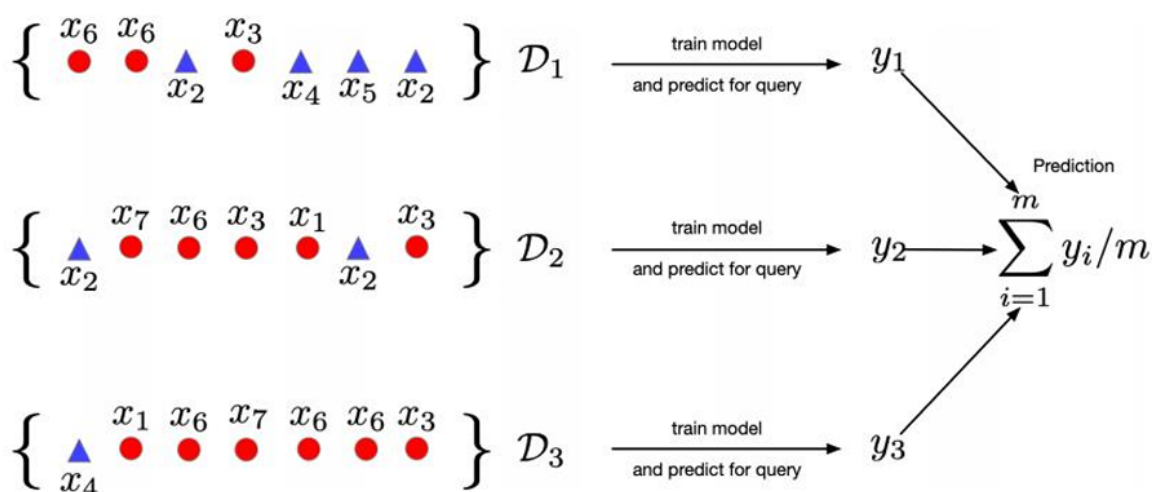
Bootstrap 是统计学中最重要的思想之一。

当数据量 n 越来越大时，从原始数据集中有放回采样得到的分布 P_D 会越来越接近真实的数据分布 P_{sample} 。

下图展示了一个 $n = 7, m = 3$ 的例子。



从同一份数据有放回地抽取7条样本，生成 3 份彼此不同的新训练集，用来训练 3 个模型，再平均/投票。



我们用这 3 组训练集分别训练了 3 个模型，我们把同一条查询样本 x 喂给每个模型，得到 3 个预测值，取平均（回归）或投票（分类）作为最终输出。

Bagging 不改变单个模型的偏差（bias），但它会“偷偷”扩大模型的假设空间，使得最终集成出来的预测函数，可能不再是原来单个模型能表示的那一类函数。

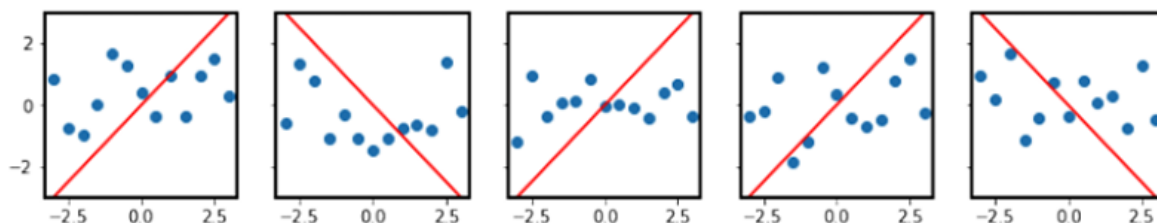
下面给出例子。

输入 x 在 $[-3, 3]$ 均匀分布，目标 t 服从标准正态噪声。

假设空间只是一条斜率受限的直线（ w 被锁在 $[-1, 1]$ ）。

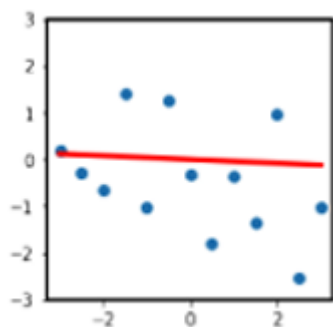
每次从真实分布采样一个数据集，用线性回归拟合出一条 wx ，斜率 w 会随数据波动而在 $[-1, 1]$ 内变

化。



把 1000 条这样的直线逐点取平均（即 Bagging 的期望预测）。

结果这条“平均直线”不再是直线。它可能带曲率，跳出了原来“只能是一条 $w x$ ”的集合 H 。



这个现象在用决策树、神经网络等非线性分类器时更明显：单棵决策树只能划轴平行线，平均后却得到任意分段边界。

1.2 二元分类

在二分类任务里，无论基分类器输出的是“概率”还是“硬投票（0/1）”，Bagging 都可以把它们的预测先平均，再统一做 0.5 阈值判断；这种“平均后再决定”的策略，往往比单个模型的准确度更高。

每个基分类器给出属于类别1的概率 $z_i \in [0, 1]$ ，

然后我们把这些概率取平均 $z_{\text{bagged}} = \frac{1}{m} \sum_{i=1}^m z_i$ ，

如果其概率超过0.5则属于类别1，

$y_{\text{bagged}} = \mathbb{I}(z_{\text{bagged}} > 0.5)$

如果我们的基分类器的结果只有0或1，即 $y_i \in \{0, 1\}$

同样可以先平均这些 0/1，再判断。

$\bar{y} = \frac{1}{m} \sum_{i=1}^m y_i$

$y_{\text{bagged}} = \mathbb{I}(\bar{y} > 0.5)$

这样“平均后再决定”的集成模型，往往比单个模型更强。

1.3 相关性的影响

如果 m 个模型的预测完全独立，方差会按 $1/m$ 线性下降。

但 Bagging 里每个模型用的数据都来自同一份原始训练集 D ，只是有放回地重采样，因此训练出的模型彼此正相关 ($\rho > 0$)。

相关性越高，平均带来的方差缩减收益越小。

$\text{Var}\left(\frac{1}{m} \sum_{i=1}^m Y_i\right) = \frac{\sigma^2}{m}(1 - \rho) + \sigma^2 \rho$ ，

其中 σ^2 ：单个模型预测的方差

ρ ：模型间预测结果的相关系数

m ：基模型数量

当 $\rho = 0$ （完全独立） $\Rightarrow$ 方差 $= \sigma^2 / m$ ，最大降幅。

当 $\rho = 1$ （完全相关） $\Rightarrow$ 方差 $= \sigma^2$ ，毫无降幅（跟用一个模型一样）。

实际 $0 < \rho < 1$ ，所以方差只能部分下降，下降比例由 ρ 决定。

1.4 随机森林 (Random Forest)

随机森林 = 把决策树做 Bagging，再加一个去相关（decorrelate）的小技巧。

每次分裂节点时，先从全部特征里随机抽 d 个候选特征，然后只在这 d 个里选最优划分。

这个 d 是超参，常用默认值：

分类： $d = \sqrt{p}$

回归： $d = p/3$ (p 为总特征数)

作用：

让各棵树“长得不一样”，降低树间相关性 ρ ，从而把 Bagging 的方差缩减效果再放大。

随机森林大概是最强黑盒算法——几乎不用调参就能跑得不错。也是 Kaggle 比赛里最常用的算法之一（尤其是结构化表格数据）。

1.5 小结

Bagging 通过平均预测结果来减少过拟合。

通过集成多个模型，即使这些模型本身已经表现不错，也可以进一步提升性能。

但是在平方误差的情况下不减少偏差。Bagging 主要通过减少方差来提高模型的泛化能力，但对模型的偏差影响不大。

分类器之间仍然存在相关性。由于所有模型都是基于同一数据集的不同子集训练的，它们之间可能存在一定的相关性，这限制了方差的进一步减少。

随机森林解决方案：增加更多的随机性。随机森林通过在每个决策节点随机选择特征来进一步减少模型之间的相关性，从而提高集成的效果。

简单的混合（所有成员权重相等）。Bagging 中的模型权重是相等的，这可能不是最优的，因为不同模型的性能可能差异很大。

如果成员非常不同（例如，不同的算法，不同的数据源等），我们通常可以通过使用有原则的加权集成方法获得更好的结果。当集成的模型差异较大时，根据模型的性能给予不同的权重可能会更有效。

接下来要讲的Boosting可以看作是一种加权集成方法，它强烈地去相关集成成员。Boosting 通过迭代地训练模型，每个新模型都试图纠正前一个模型的错误，从而实现模型之间的去相关，并根据模型的表现给予不同的权重。

2. Boosting

顺序训练分类器，每次训练时专注于前一个分类器分类错误的样本，通过这种方式逐步提升整体模型的性能。

由于每个模型都专注于不同的错误样本，它们的预测结果之间的相关性降低，这有助于提高集成模型的泛化能力。

为了专注于特定样本，Boosting 使用加权的训练集。每个样本在训练集中的权重会根据其被前一个模型分类的正确与否进行调整。被错误分类的样本权重会增加，从而在后续模型中得到更多的关注。

传统的错误分类率 $\frac{1}{N} \sum_{n=1}^N \mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$ 将每个训练样本视为等权重。这里， $\mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$ 是指示函数，当模型 h 对样本 $x^{(n)}$ 预测不等于真实标签时，其值为1，否则为0。

我们可以为不同的样本分配不同的成本（或权重）来学习分类器。

分类器会“更努力”地学习那些具有更高成本的样本。

于是我们将原来的成本函数 $\sum_{n=1}^N w^{(n)} \mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$ 修改为 $\sum_{n=1}^N w^{(n)} = 1$ ，这里， $w^{(n)}$ 是样本 n 的权重。

通常要求每个权重 $w^{(n)} > 0$ 且所有权重之和为1。

2.1 AdaBoost 算法

AdaBoost 的关键步骤如下：

1. 在每次迭代中，通过给被错误分类的样本（即数据点）分配更大的权重来重新加权训练样本。
2. 基于重新加权的样本训练一个新的基础分类器。

3. 将其以适当的权重添加到分类器集成中。
4. 多次重复这个过程。

基础分类器的要求：

- 需要最小化加权误差。
- 集成可能会变得非常大，因此基础分类器必须快速。事实证明，任何所谓的弱学习器/分类器都可以。

单独来看，弱学习器可能具有高偏差（欠拟合）。通过让每个分类器专注于之前的错误，AdaBoost 减少了偏差。

2.1.1 弱学习器 (Weak Learner)

弱学习器是一种学习算法，它输出的假设（例如分类器）的性能略好于随机猜测。例如，在二分类问题中，它以0.51的概率正确预测标签。

我们对计算效率高的弱学习器感兴趣。

例如决策树是一种常见的弱学习器，因为它可以快速地进行训练和预测。

更简单的是决策桩（Decision Stump），它是一种只有一个分裂的决策树。

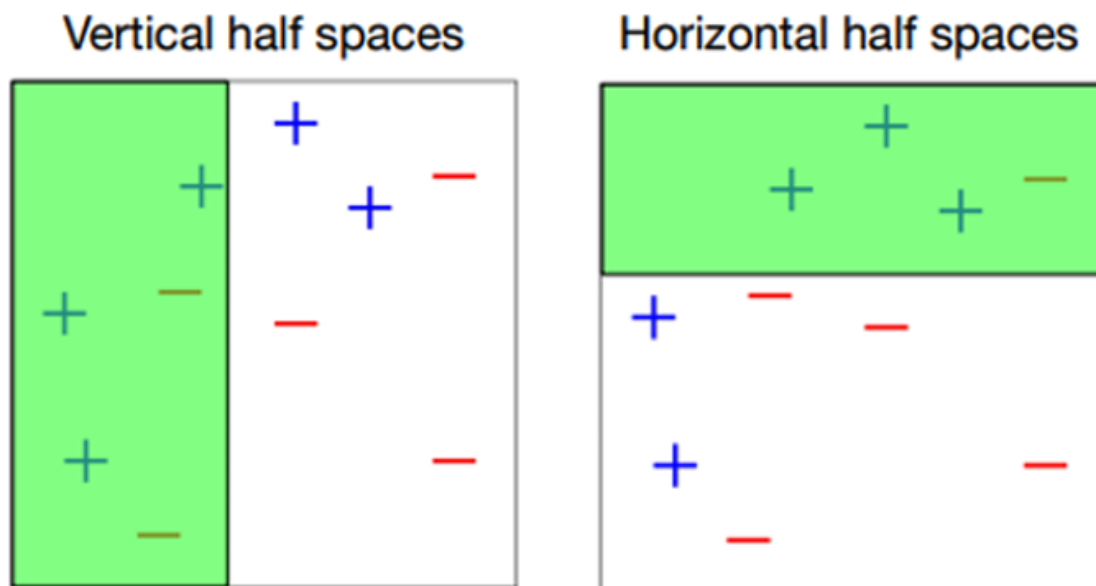
正式定义：弱可学习性（weak learnability）的正式定义涉及到一些量化指标，例如“对于数据的任何分布”，并且其保证仅以概率形式成立。

这意味着在理论上，弱学习器能够在一定的概率下，对于任何数据分布，找到一个略好于随机猜测的分类器。

2.1.1.1 决策桩 (Decision Stumps)

决策桩是最简单的决策树，由水平和垂直的半空间组成。

如下图所示。



左边是垂直半空间（Vertical half spaces）：基于特征值的垂直分割，例如 $x_2 \leq 2$ 或 $x_2 > 2$ 。

右边是水平半空间（Horizontal half spaces）：基于特征值的水平分割，例如 $x_1 \leq 2$ 或 $x_1 > 2$ 。

单个弱分类器无法使训练误差变得很小。

但如果能保证它的表现略好于随机猜测，即分类器 h 根据给定的权重 $\mathbf{w} = w_1, \dots, w_N$ 的加权误差最多是 $\frac{1}{2} - \gamma$ （其中 $\gamma > 0$ ），那么使用AdaBoost结合这些弱分类器可以给我们一个通用函数逼近器。

上次我们使用信息增益作为分裂标准。在使用AdaBoost的决策桩时，我们通常使用“GINI不纯度”，这（大致上）选择直接最小化误差的分裂。

GINI不纯度是一种衡量数据集不纯度的方法，它倾向于选择那些能够最大程度减少分类错误的分裂。

我们回顾之前学习的知识。

输入数据集 $\mathcal{D}_N$ 包含 N 个样本，每个样本由特征向量 $\mathbf{x}^{(n)}$ 和对应的标签 $t^{(n)}$ 组成。这里标签的取值是 -1 或 +1 与之前的 0, 1 不同。但这只是为了方便记号，两者在本质上是等价的。

$\mathcal{D}_N = \{\mathbf{x}^{(n)}, t^{(n)}\}_{n=1}^N$ where $t^{(n)} \in \{-1, +1\}$

分类器或假设 h 是一个函数，它将输入特征向量 $\mathbf{x}$ 映射到输出标签 -1 或 +1。

$h: \mathbf{x} \rightarrow \{-1, +1\}$

0-1 损失函数用于衡量分类器的预测结果与真实标签之间的差异。

$\mathbb{I}[h(\mathbf{x}^{(n)}) \neq t^{(n)}] = \frac{1}{2}(1 - h(\mathbf{x}^{(n)}) \cdot t^{(n)})$

其中 $\mathbb{I}[h(\mathbf{x}^{(n)}) \neq t^{(n)}]$ 是指示函数，当预测结果与真实标签不同时，其值为 1，否则为 0。

等号后面的式子也可以用来计算 0-1 损失，其中 $h(\mathbf{x}^{(n)}) \cdot t^{(n)}$ 是预测标签与真实标签的点积。

2.1.2 AdaBoost 算法的具体步骤

输入：输入数据集 $\mathcal{D}_N$ 包含 N 个样本，每个样本由特征向量 $\mathbf{x}^{(n)}$ 和对应的标签 $t^{(n)}$ 组成。

$\mathcal{D}_N = \{\mathbf{x}^{(n)}, t^{(n)}\}_{n=1}^N$ where $t^{(n)} \in \{-1, +1\}$

一个返回分类器 h 的分类过程，例如最佳决策桩，从一个分类器集合 H 中选择，例如所有可能的决策桩。

以及迭代次数 T 。

输出：最终的分分类器。

步骤：

1. 初始化样本权重：

$w^{(n)} = \frac{1}{N}$ 对于所有 $n = 1, \dots, N$

2. 对于 $t = 1, \dots, T$

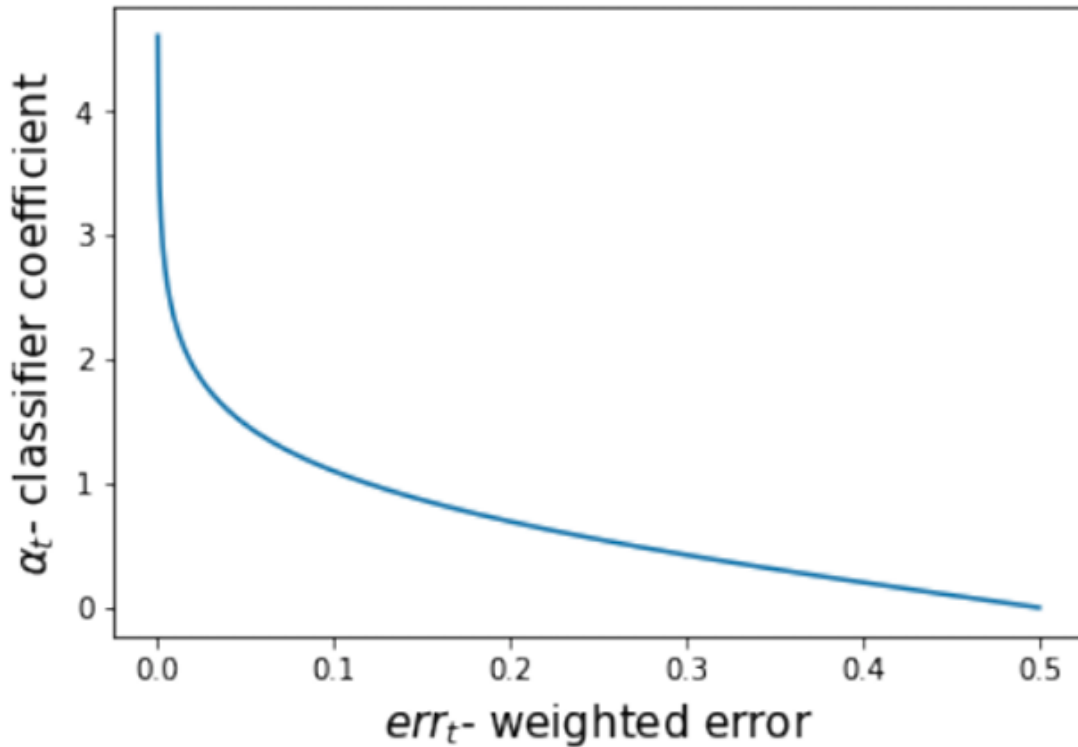
- 使用加权数据训练一个分类器 h_t : $h_t \leftarrow \arg \min_{h \in \mathcal{H}} \sum_{n=1}^N w^{(n)} \mathbb{I}\{h(\mathbf{x}^{(n)}) \neq t^{(n)}\}$
- 计算加权错误 err_t : $\text{err}_t = \frac{\sum_{n=1}^N w^{(n)} \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\}}{\sum_{n=1}^N w^{(n)}}$
- 计算分类器系数 α_t : $\alpha_t = \frac{1}{2} \log \frac{1 - \text{err}_t}{\text{err}_t}$ (其中 $\text{err}_t \in (0, \infty)$)
- 更新数据权重 $w^{(n)}$: $w^{(n)} \leftarrow w^{(n)} \exp(-\alpha_t t^{(n)} h_t(\mathbf{x}^{(n)}))$ 等价于 $w^{(n)} \exp(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\})$

3. 返回最终的分分类器 $H(\mathbf{x})$: $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$, 它是所有弱分类器的加权符号函数。

2.1.2.1 AdaBoost 算法中权重调整

我们最后最终分类器 $H(\mathbf{x})$: $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$ 是所有弱分类器 $h_t(\mathbf{x})$ 的加权和的符号函数, 其中 α_t 是第 t 个弱分类器的系数。

这个系数 $\alpha_t = \frac{1}{2} \log \frac{1-\text{err}_t}{\text{err}_t}$ 由第 t 个弱分类器的错误率 err_t 决定。



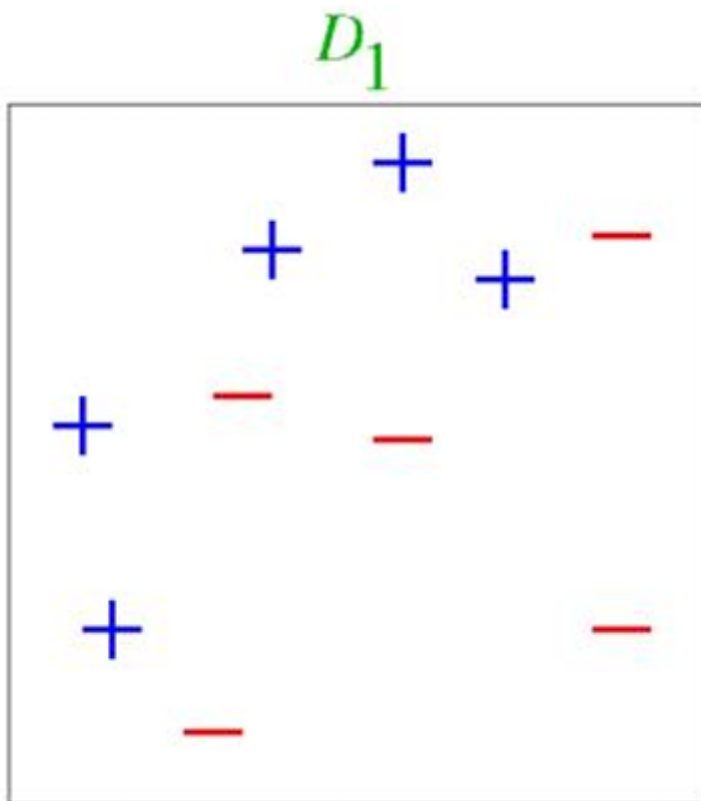
- 当 err_t 接近 0 时, α_t 较大, 这意味着分类器在训练数据上的错误率很低, 因此它在最终分类器中的影响较大。
- 当 err_t 接近 0.5 时, α_t 较小, 这意味着分类器在训练数据上的错误率较高, 因此它在最终分类器中的影响较小。

同样样本权重 $w^{(n)}$ 根据第 t 个弱分类器对样本 $\mathbf{x}^{(n)}$ 的分类结果进行更新:

$$w^{(n)} \leftarrow w^{(n)} \exp(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\})$$

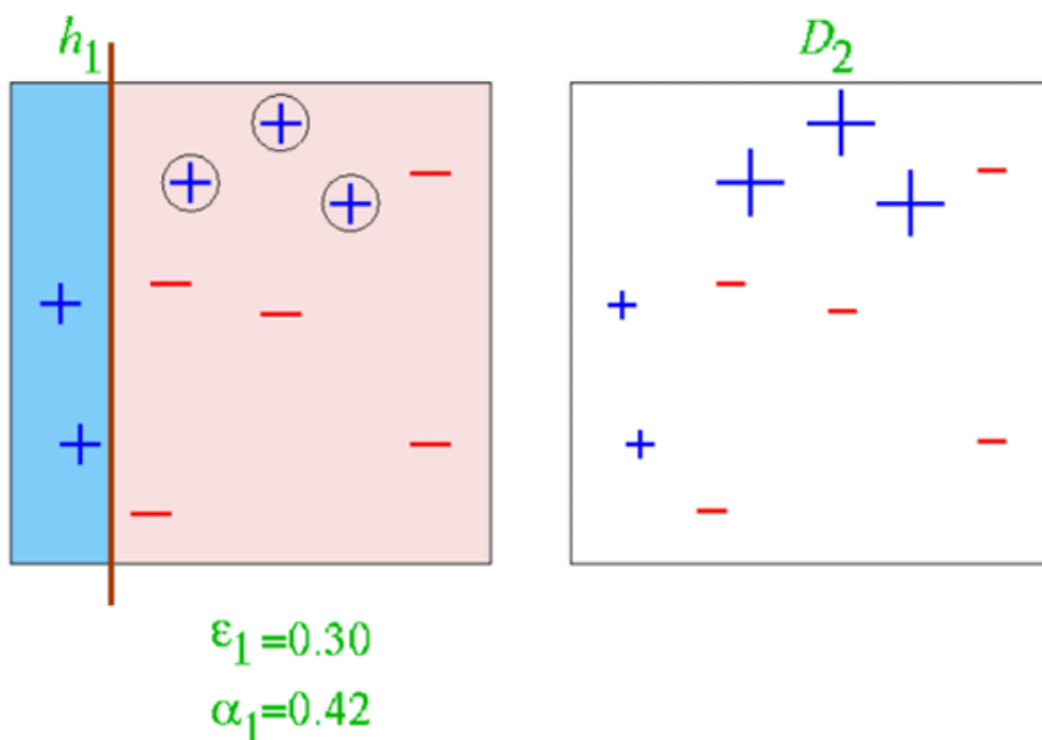
- 如果 $h_t(\mathbf{x}^{(n)}) \neq t^{(n)}$ (即样本被错误分类), 则权重增加, 这意味着在后续迭代中, 这个样本会被赋予更多的关注。
- 如果 $h_t(\mathbf{x}^{(n)}) = t^{(n)}$ (即样本被正确分类), 则权重减少, 这意味着在后续迭代中, 这个样本会被赋予较少的关注。

2.1.3 AdaBoost算法的示例



这是用于训练的数据集，标记为 D_1 。

第一轮迭代:



左图显示了第一个弱分类器 h_1 的决策边界，它将数据分为两个区域（蓝色和粉色），每个区域内的样本被分类为同一类别。

右图显示了根据第一个弱分类器 h_1 的错误分类结果重新加权后的数据集 D_2 ，其中被错误分类的样本（位于决策边界另一侧的样本）在后续迭代中将被赋予更高的权重。

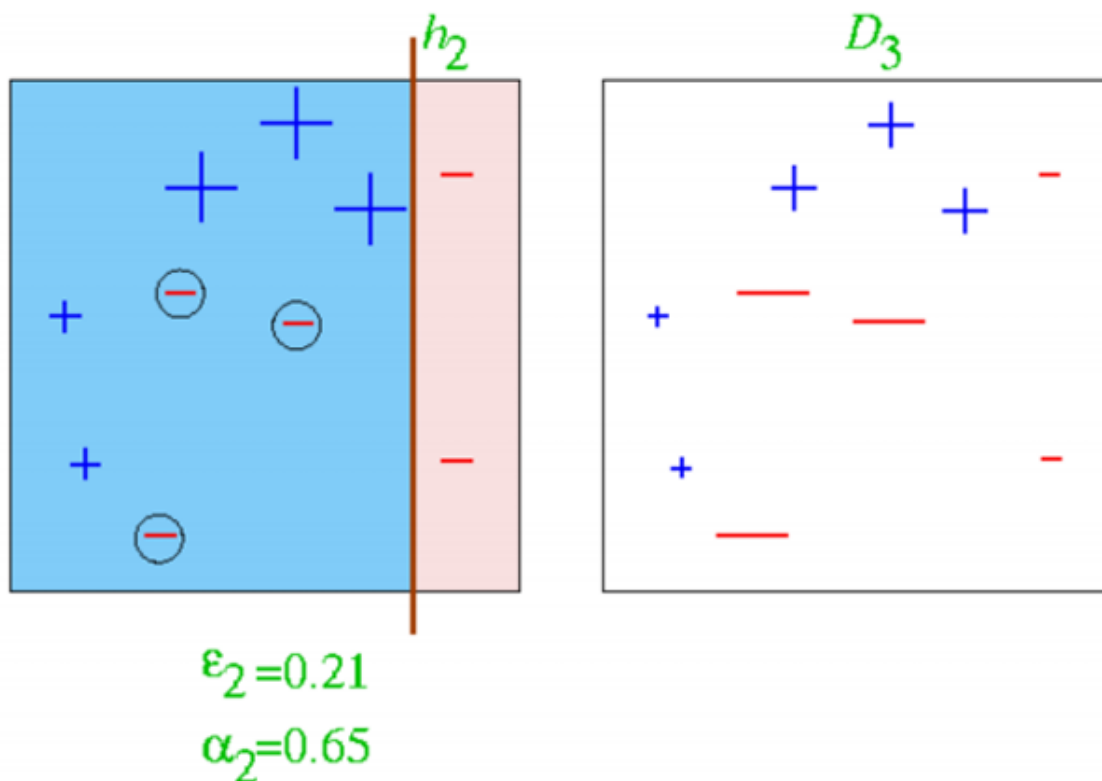
第一个弱分类器的错误率为 $\epsilon_1 = 0.30$

第一个弱分类器的权重为 $\alpha_1 = \frac{1}{2} \log \frac{1-\epsilon_1}{\epsilon_1} = 0.42$

我们现在一步步执行：

- 权重初始化 $\mathbf{w} = (\frac{1}{10}, \dots, \frac{1}{10})$, 表示所有样本初始权重相等。
- 训练分类器: 使用初始权重 $\mathbf{w}$ 训练第一个弱分类器 h_1 。
- 计算错误率: $\text{err}_1 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_1(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{10} w_i} = \frac{3}{10}$, 表示有 3 个样本被错误分类。
- 计算分类器系数: $\alpha_1 = \frac{1}{2} \log \frac{1-\text{err}_1}{\text{err}_1} = \frac{1}{2} \log \frac{1}{0.3} - 1 \approx 0.42$ 。
- 最终分类器: $H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}))$, 最终分类器是第一个弱分类器的加权结果。

第二轮迭代:



左图显示了第二个弱分类器 h_2 的决策边界, 它专注于第一轮中被错误分类的样本 (蓝色区域)。

右图显示了根据第一个弱分类器 h_2 的错误分类结果重新加权后的数据集 D_3 , 其中被错误分类的样本 (位于决策边界另一侧的样本) 在后续迭代中将被赋予更高的权重。

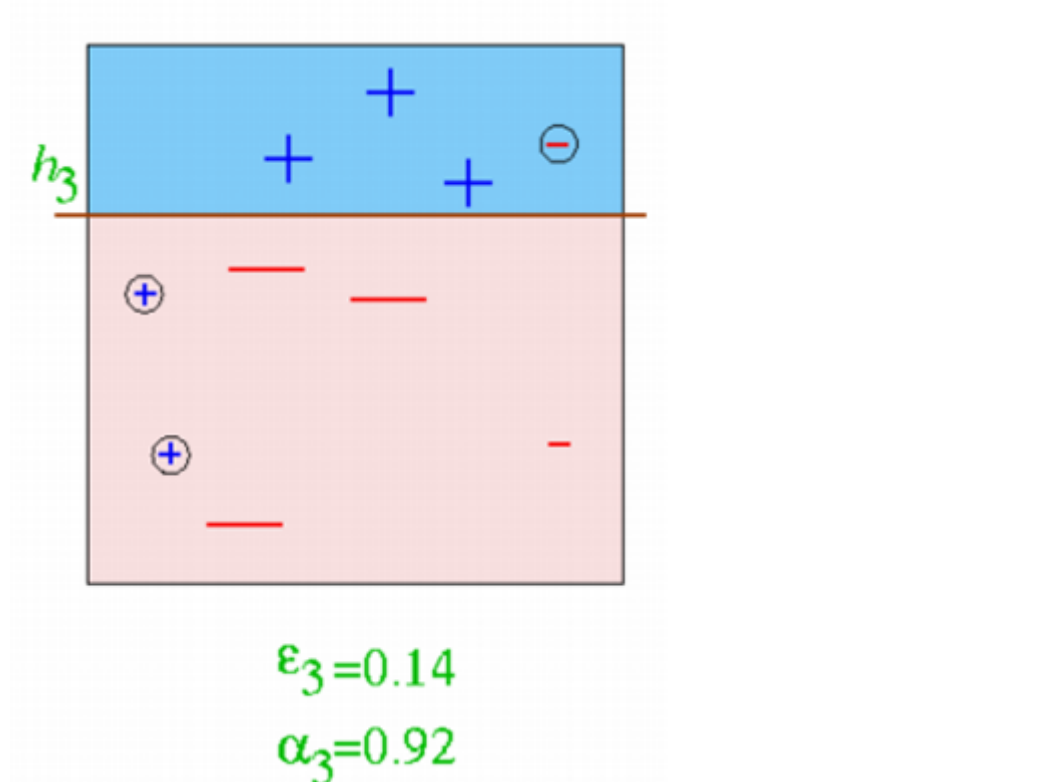
第二个弱分类器的错误率为 $\epsilon_2 = 0.21$ 。

第二个弱分类器的权重为 $\alpha_2 = 0.65$ 。

我们现在一步步执行:

- 权重更新 $\mathbf{w} = \text{updated weights}$, 所有样本权重根据第一轮的结果进行了更新。
- 使用更新后的权重 $\mathbf{w}$ 训练第二个弱分类器 h_2 。
- 计算错误率: $\text{err}_2 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_2(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{10} w_i} = 0.21$ 。
- 计算分类器系数: $\alpha_2 = \frac{1}{2} \log \frac{1-\text{err}_2}{\text{err}_2} = \frac{1}{2} \log \frac{1}{0.21} \approx 0.66$ 。
- 最终分类器: $H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}))$, 最终分类器是前两个弱分类器的加权结果。

第三轮迭代：



上图显示了第三个弱分类器 h_3 的决策边界，它专注于第二轮中被错误分类的样本。

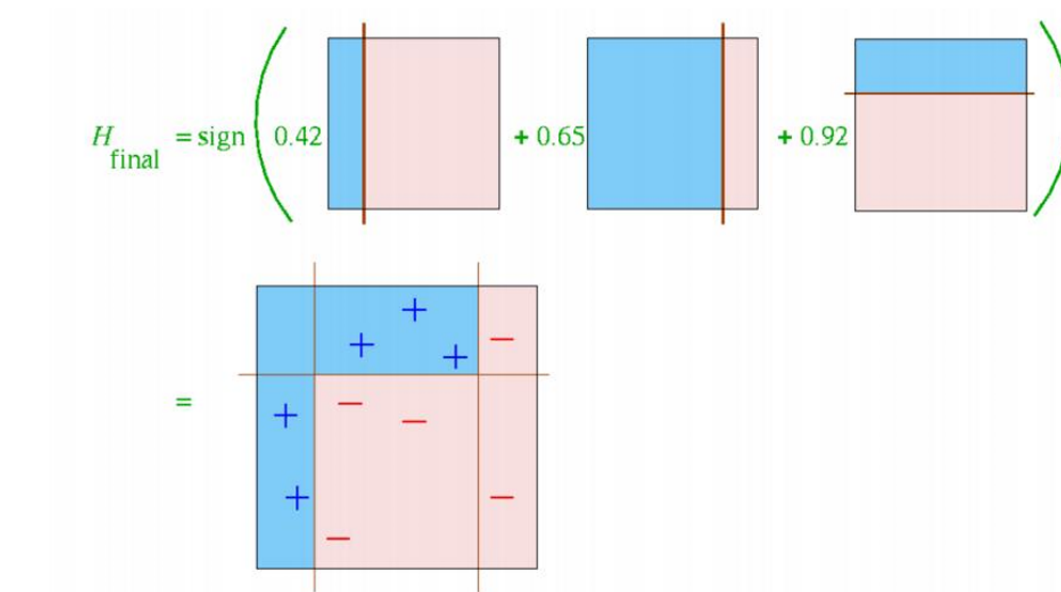
第三个弱分类器的错误率为 $\epsilon_3 = 0.14$ 。

第三个弱分类器的权重为 $\alpha_3 = 0.92$ 。

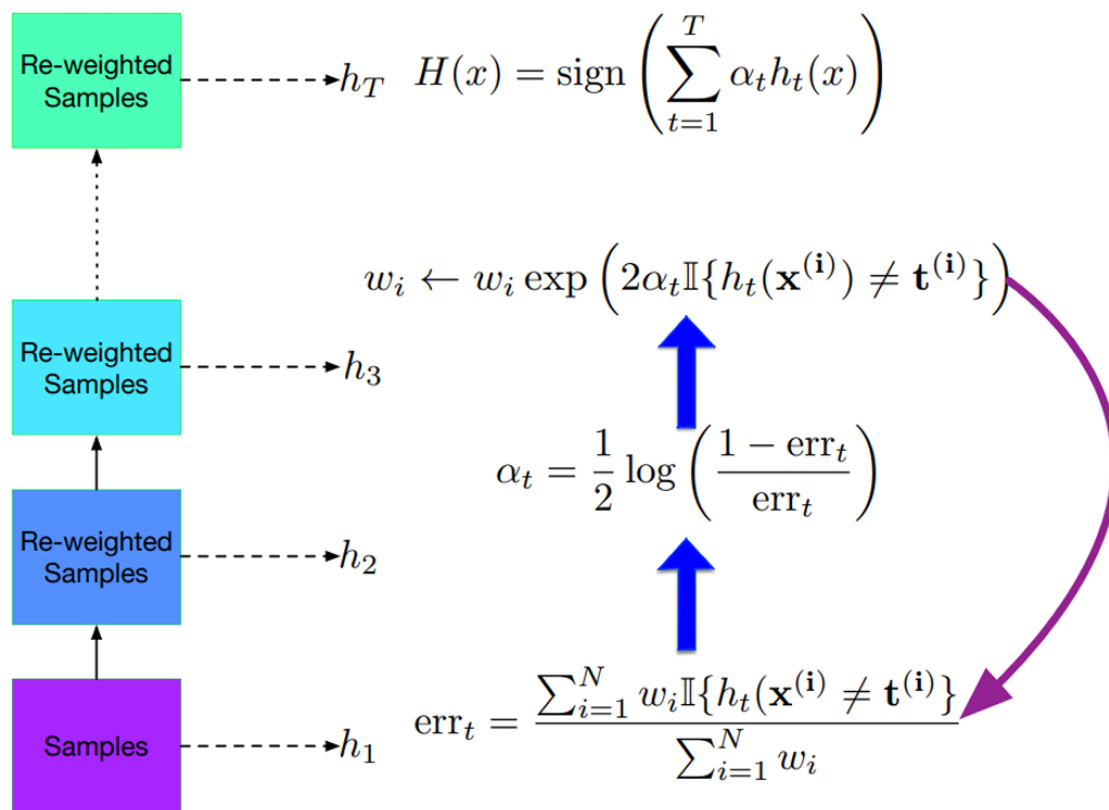
我们现在一步步执行：

- 权重更新 $\mathbf{w}$ = updated weights, 所有样本权重根据第二轮的结果进行了更新。
- 使用更新后的权重 $\mathbf{w}$ 训练第三个弱分类器 h_3 。
- 计算错误率: $\text{err}_3 = \frac{\sum_{i=1}^{10} w_i \mathbb{I}\{h_3(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^{10} w_i} = 0.14$ 。
- 计算分类器系数: $\alpha_3 = \frac{1}{2} \log \frac{1 - \text{err}_3}{\text{err}_3} = \frac{1}{2} \log \frac{1}{0.14} \approx 0.91$ 。
- 最终分类器: $H(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x}))$, 最终分类器是前三个弱分类器的加权结果。

所以我们得到最终的分类器如下图所示。

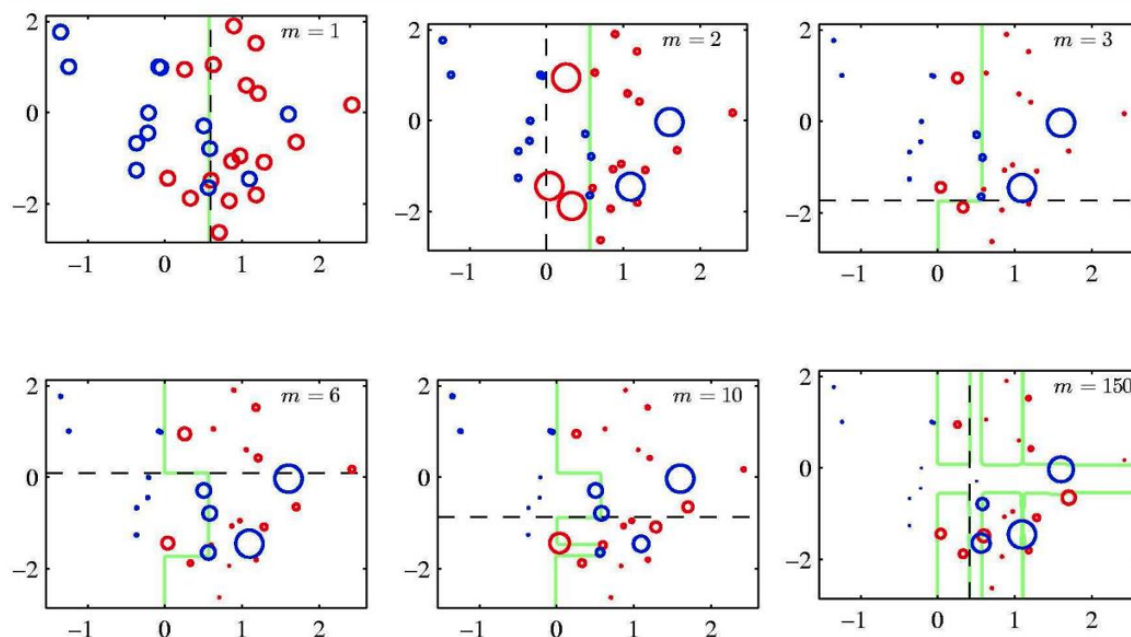


$$H_{\text{final}} = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x}))$$



所以我们的步骤就是用弱分类器分类，然后计算错误率，然后根据错误率计算弱分类器的权重，再根据弱分类器的权重计算样本权重。最后将所有弱分类器加权和得到最终分类器。

下图又展现了一个例子。



每个图展示了在不同数量的弱分类器 (m) 训练完成后，最近一个弱分类器的决策边界 (虚线黑色)，以及集成分类器的决策边界 (绿色)。

随着弱分类器数量的增加，集成分类器的决策边界变得更加复杂和精确，能够更好地捕捉数据中的模式。

2.1.4 最小化训练误差

下面的定理展示了 AdaBoost 算法的一个重要性质：在每次迭代中，如果每个弱学习器都比随机猜测好 γ ，那么最终集成模型的训练误差会以几何级数快速下降。这种快速的收敛性是 AdaBoost 算法的一个显著优点，使其在实践中非常有效。

定理：假设在 AdaBoost 的每次迭代中，WeakLearn 返回的假设（分类器）的错误率 $\text{err}_t \leq \frac{1}{2} - \gamma$ 对于所有 $t = 1, \dots, T$ 且 $\gamma > 0$ 。输出假设 $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$ 的训练误差最多为：

$$L_N(H) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}\{H(\mathbf{x}^{(i)}) \neq t^{(i)}\} \leq \exp(-2\gamma^2 T)$$

其中 $L_N(H)$ 是输出假设 H 在训练集上的错误率。

这是在简化的假设下得出的，即每个弱学习器都比随机预测器好 γ 。

这被称为几何级数收敛，它非常快。

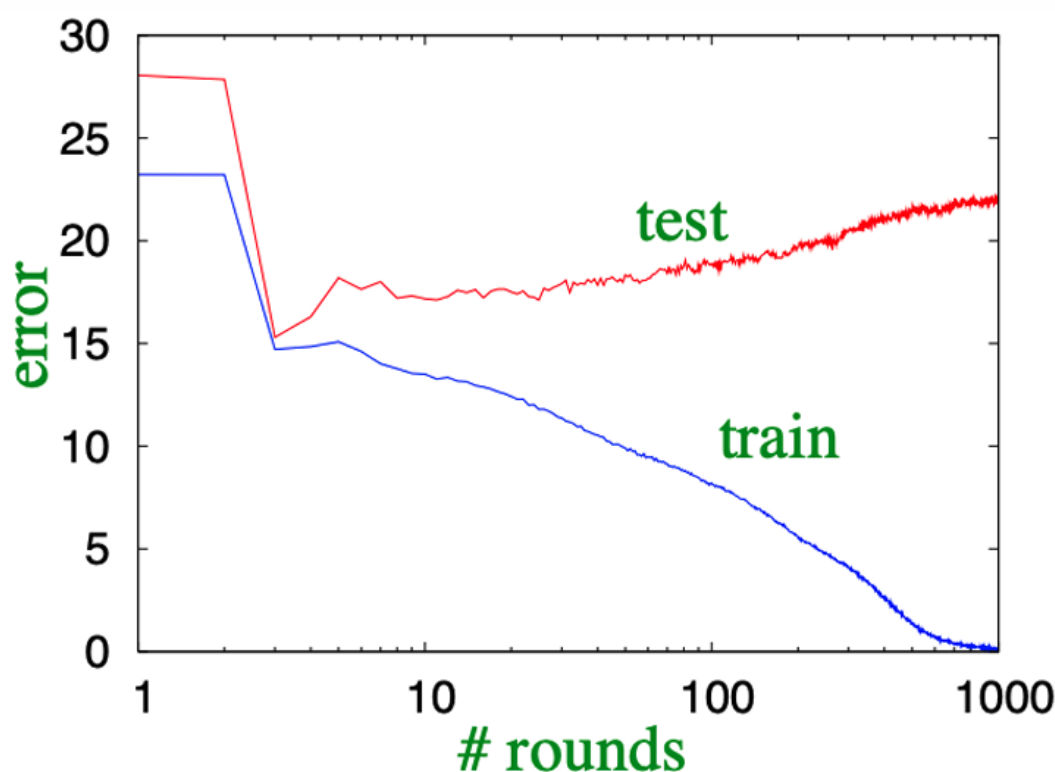
2.1.5 AdaBoost 算法的泛化误差 (generalization error)

AdaBoost 的训练误差（损失）收敛到零。那么 H 的测试误差呢？

换句话说，虽然 AdaBoost 的训练误差可以降到很低，甚至接近于零，但这是否意味着它在测试集上的表现也同样优秀？

随着我们添加更多的弱分类器，整体分类器 H 变得更加“复杂”。我们预计更复杂的分类器会过拟合。

如果运行 AdaBoost 足够长的时间，它实际上可能会过拟合。

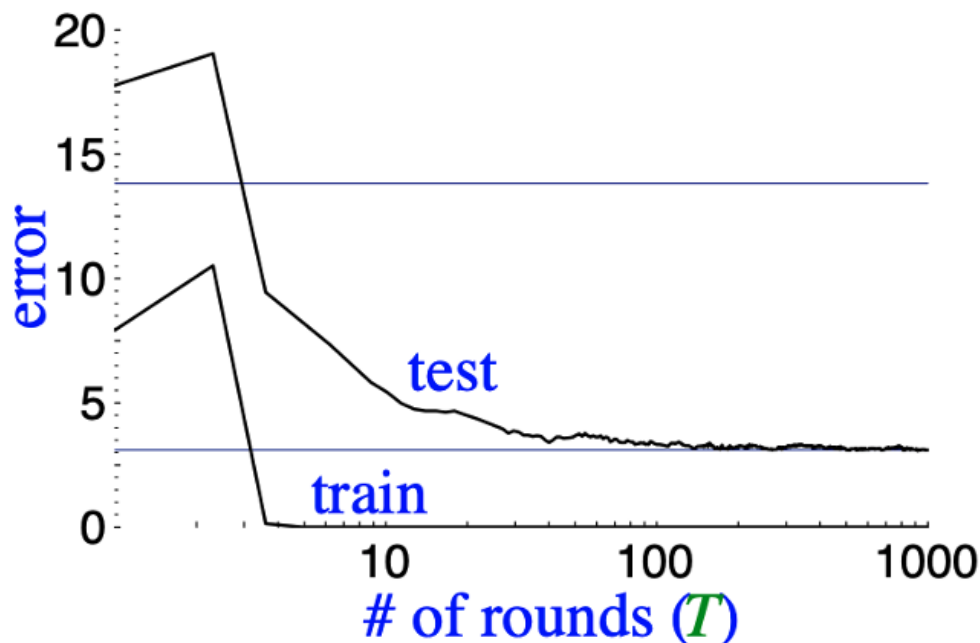


图中展示了训练误差和测试误差随迭代次数（rounds）的变化趋势：

蓝色曲线表示训练误差，随着迭代次数的增加，训练误差逐渐降低，最终趋近于零。

红色曲线表示测试误差，开始时测试误差也降低，但在某个点之后开始上升，显示出过拟合的迹象。

有时候，即使训练误差已经降到零，测试误差仍然会继续下降。



图中展示了训练误差和测试误差随迭代次数 (rounds) 的变化趋势：

蓝色曲线表示训练误差，随着迭代次数的增加，训练误差迅速下降并趋近于零。

红色曲线表示测试误差，开始时测试误差也下降，但在训练误差降到零后，测试误差继续下降，然后趋于平稳。

在接下来的讨论中，我们将介绍一种不同的方法或理论框架来解释 AdaBoost 的工作原理，这将解答它如何在训练误差为零后仍然能够改善测试误差的原因。

2.1.6 作为拟合加性模型 (Additive Models)

接下来，我们将 AdaBoost 解释为拟合加性模型的一种方式。

考虑一个假设类 $\mathcal{H}$ ，其中每个 h_i 映射到 $\{-1, +1\}$ ，即 $h_i \in \mathcal{H}$ 。这些是“弱学习器”，在这种情况下，它们也被称为基 (bases)。

一个包含 m 项的加性模型由下式给出：

$$H_m(\mathbf{x}) = \sum_{i=1}^m \alpha_i h_i(\mathbf{x}), \text{ 其中 } (\alpha_1, \dots, \alpha_m) \in \mathbb{R}^m.$$

注意到我们正在取基分类器 $h_i(x)$ 的线性组合，就像在提升中一样。还要注意与我们在线性回归和神经网络中看到的特征映射（或基扩展）的联系。

加性模型 (Additive Models) 的分阶段训练方法是 AdaBoost 算法背后的原理之一。

阶梯式训练 (Stagewise Training) 是一种贪心算法，用于拟合加性模型。

步骤如下：

1. 初始化模型 H_0 为 0，即最开始的模型不包含任何分类器。

2. 从 $m = 1$ 迭代到 T ：

- 计算第 m 个假设 H_m ，即在前一个加性模型 H_{m-1} 的基础上添加一个新的弱分类器 h_m 和对应的权重 α_m 。

$$H_m = H_{m-1} + \alpha_m h_m$$

- 找到使得损失函数 $\mathcal{L}$ 最小的弱分类器 h_m 和权重 α_m 。这里 $\mathcal{L}$ 是损失函数， N 是样本数量， $t^{(i)}$ 是第 i 个样本的真实标签。

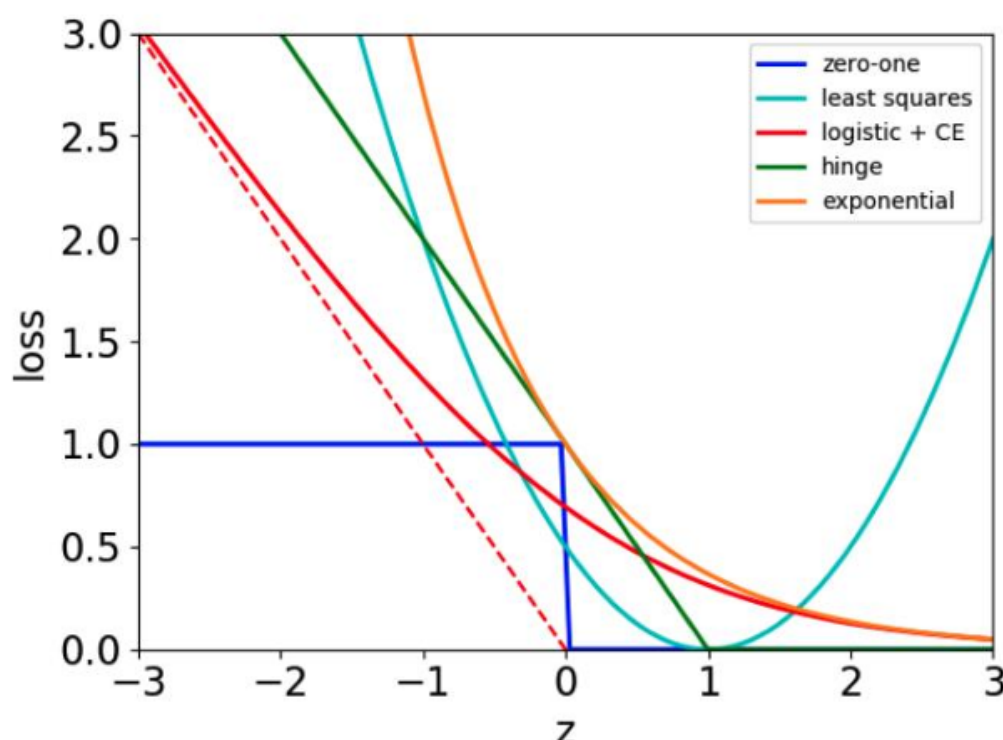
$$(h_m, \alpha_m) \leftarrow \arg \min_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N \mathcal{L}(H_{m-1}(\mathbf{x}^{(i)}) + \alpha h(\mathbf{x}^{(i)}), t^{(i)})$$

- 将找到的弱分类器 h_m 及其权重 α_m 添加到加性模型中，更新模型为 $H_m = H_{m-1} + \alpha_m h_m$ 。

2.1.6.1 指数损失函数

指数损失函数定义为 $\mathcal{L}_E(z, t) = \exp(-tz)$ ，其中 z 是模型预测值， t 是真实标签。

下图展示了不同损失函数随预测值 z 的变化曲线：



zero-one (蓝色实线)：0-1损失，常用于分类问题。

least squares (青色实线)：最小二乘损失，常用于回归问题。

logistic + CE (红色虚线)：逻辑回归损失加交叉熵损失，常用于二分类问题。

hinge (绿色实线)：合页损失，AdaBoost中常用的损失函数。

exponential (橙色实线)：指数损失，强调了对错误预测的惩罚随着预测错误的增大而指数增长。

所以指数损失函数在预测值与真实标签相差较大时，损失增长速度更快，这使得模型在训练时更加关注那些难以分类的样本。

我们现在详细看一下加性模型的分阶段训练是如何进行的。

指数损失函数定义为 $\mathcal{L}_E(z, t) = \exp(-tz)$ ，其中 z 是模型预测值， t 是真实标签。

找到使得指数损失函数最小化的弱分类器 h_m 和权重 α_m 。

$$(h_m, \alpha_m) \leftarrow \arg \min_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N \exp(-[H_{m-1}(\mathbf{x}^{(i)}) + \alpha h(\mathbf{x}^{(i)})]t^{(i)})$$

将损失函数展开为两部分的乘积。

$$= \sum_{i=1}^N \exp(-H_{m-1}(\mathbf{x}^{(i)})t^{(i)}) \exp(-\alpha h(\mathbf{x}^{(i)})t^{(i)})$$

进一步简化为样本权重 $w_i^{(m)}$ 与指数项的乘积。

$$= \sum_{i=1}^N w_i^{(m)} \exp(-\alpha h(\mathbf{x}^{(i)})t^{(i)})$$

这里我们定义 $w_i^{(m)} \triangleq \exp(-H_{m-1}(\mathbf{x}^{(i)})t^{(i)})$ 不依赖于 h, α 。

我们想要解决以下最小化问题： $(h_m, \alpha_m) \leftarrow \arg \min_{h \in \mathcal{H}, \alpha} \sum_{i=1}^N w_i^{(m)} \exp(-\alpha h(\mathbf{x}^{(i)})t^{(i)})$

这里， (h_m, α_m) 是第 m 个弱分类器和对应的权重， $\mathcal{H}$ 是弱分类器的集合， $w_i^{(m)}$ 是第 m 轮迭代中第 i 个样本的权重， α 是权重参数， $h(\mathbf{x}^{(i)})$ 是弱分类器对第 i 个样本的预测， $t^{(n)}$ 是第 i 个样本的真实标签。

前面我们推导过样本权重和指数损失的关系：

$$w^{(n)} \exp(-\alpha_t h_t(\mathbf{x}^{(n)})t^{(n)}) \propto w^{(n)} \exp(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\})$$

因此，对于 h_m ，上述最小化等价于：

$$h_m \leftarrow \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \exp(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\})$$

进一步简化为：

$$= \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} (\exp(2\alpha_t \mathbb{I}\{h_t(\mathbf{x}^{(n)}) \neq t^{(n)}\}) - 1)$$

通过除以 $\exp(2\alpha_t) - 1$ 得到：

$$= \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_t(\mathbf{x}^{(i)}) \neq t^{(i)}\}$$

这意味着 h_m 是加权 0/1 损失的最小化器。

现在我们已经获得了第 m 个弱分类器 h_m ，我们可以将其代入我们的指数损失目标函数并求解 α_m 。

$$\text{推导结果为 } \alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$$

其中 err_m 是加权分类错误率：

$$\text{err}_m = \frac{\sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_m(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i^{(m)}}$$

这是指在第 m 轮迭代中，弱分类器 h_m 错误分类的样本的权重之和除以所有样本权重之和。

我们现在可以找到下一轮迭代的更新后的样本权重：

$$w_i^{(m+1)} = \exp \left(-H_m(\mathbf{x}^{(i)}) t^{(i)} \right)$$

更新后的权重 $w_i^{(m+1)}$ 由当前模型 H_m 对第 i 个样本的预测值和真实标签 $t^{(i)}$ 计算得出。

$$= \exp \left(-[H_{m-1}(\mathbf{x}^{(i)}) + \alpha_m h_m(\mathbf{x}^{(i)})] t^{(i)} \right)$$

这里 $H_{m-1}(\mathbf{x}^{(i)})$ 是前一轮迭代的加性模型预测值， α_m 是第 m 个弱分类器的权重， $h_m(\mathbf{x}^{(i)})$ 是第 m 个弱分类器对第 i 个样本的预测值。

$$= \exp \left(-H_{m-1}(\mathbf{x}^{(i)}) t^{(i)} \right) \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right)$$

将上式展开，得到前一轮模型预测和当前弱分类器预测的乘积。

$$= w_i^{(m)} \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right)$$

最终表达式显示，更新后的权重是当前权重 $w_i^{(m)}$ 乘以 $\exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right)$ ，这反映了当前弱分类器对每个样本预测的影响。

2.1.7 AdaBoost 算法小结

总结起来，我们得到了加性模型 $H_m(\mathbf{x}) = \sum_{i=1}^m \alpha_i h_i(\mathbf{x})$ ，其中：

$$h_m \leftarrow \arg \min_{h \in \mathcal{H}} \sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h(\mathbf{x}^{(i)}) \neq t^{(i)}\}$$

找到使得加权错误率最小的弱分类器 h_m 。

$$\alpha = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$$

计算第 m 个弱分类器的权重 α ，其中 err_m 是第 m 个弱分类器的错误率。

$$\text{err}_m = \frac{\sum_{i=1}^N w_i^{(m)} \mathbb{I}\{h_m(\mathbf{x}^{(i)}) \neq t^{(i)}\}}{\sum_{i=1}^N w_i^{(m)}}$$

计算第 m 个弱分类器的加权错误率 err_m 。

$$w_i^{(m+1)} = w_i^{(m)} \exp \left(-\alpha_m h_m(\mathbf{x}^{(i)}) t^{(i)} \right)$$

更新第 i 个样本在第 $m + 1$ 轮迭代中的权重 $w_i^{(m+1)}$

2.2 小结

Boosting 通过生成一系列弱分类器的集成来减少偏差。

这里的“弱分类器”指的是性能略优于随机猜测的分类器，它们是算法中的基本构建块。

每个分类器都被训练来减少前一个集成的错误。

它对过拟合相当有抵抗力，尽管它也可能过拟合。

3. 集成学习

我们今天学习了集成学习的两种主要方法：Boosting（提升法）和 Bagging（装袋法）。

集成方法通过组合多个分类器来提高性能。

Boosting（提升法）：

- 提升法通过逐步纠正前一个模型的错误来减少模型的偏差。
- 由于模型是顺序训练的，集成的方差增加，可能导致过拟合。

- 模型是顺序训练的，每个模型依赖于前一个模型的结果。
- 集成中的元素之间存在高度依赖性。

Bagging（装袋法）：

- 装袋法通过并行训练多个模型来减少方差，大的集成不会引发过拟合。
- 装袋法对模型的偏差影响不大。
- 模型可以并行训练，因为它们是独立于彼此的。
- 目标是最小化集成元素之间的相关性，以进一步减少方差。

因此我们可以得到。

方法	主要目标	偏差（Bias）	方差（Variance）	风险
Bagging	降低方差	几乎不变	✅ 显著降低	欠拟合风险小
Boosting	降低偏差	✅ 显著降低	可能略升	过拟合风险大